

软件开发的范式革命

在代码诞生之前，完成最终产品级的测试

胥克谦

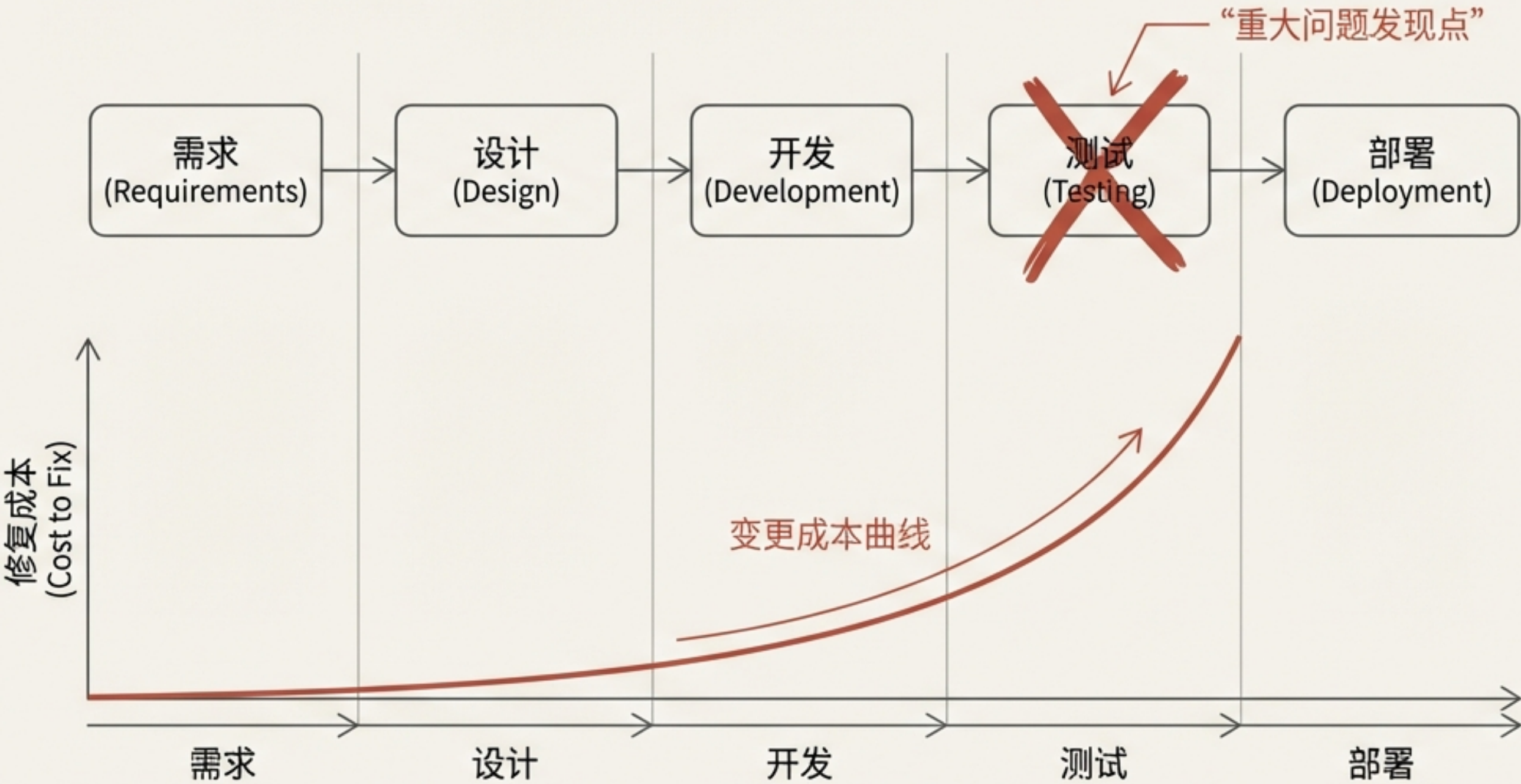
微信: xukeqian

传统开发流程：昂贵的“后知后觉”

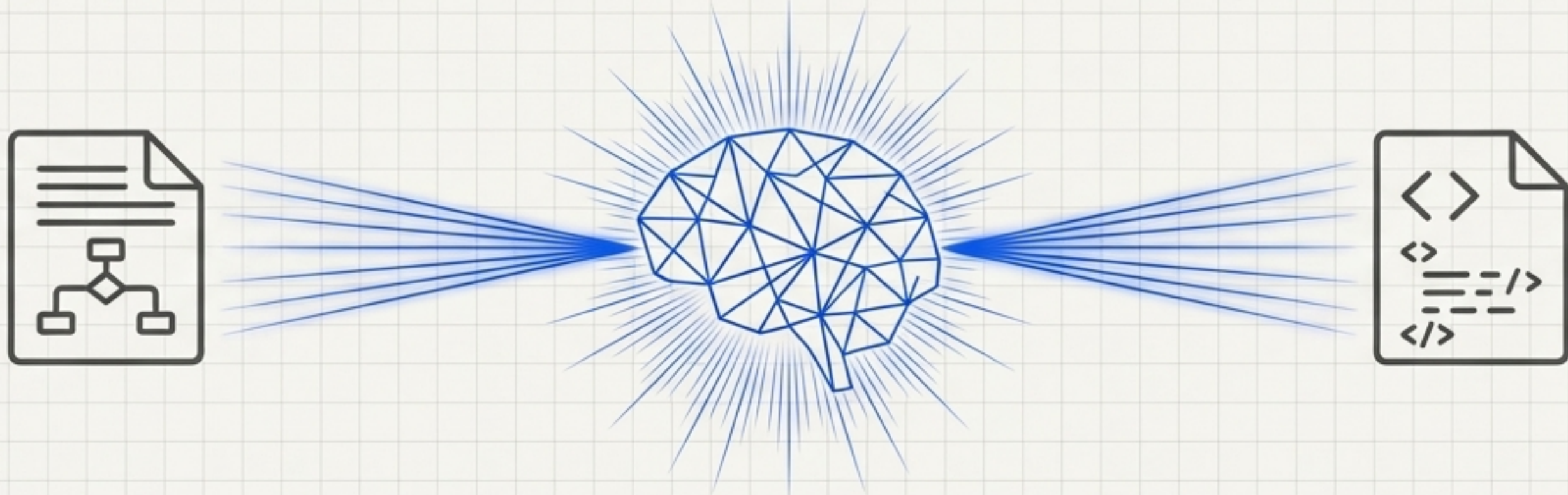
我们都熟悉这个流程：

从需求、设计、开发到测试，问题发现得越晚，修复的成本和代价就呈指数级增长。

最终的用户验收测试（UAT）环节，往往是缺陷和设计问题的“重灾区”，导致大量的返工和延期。



AI原生的核心洞察：文档即代码

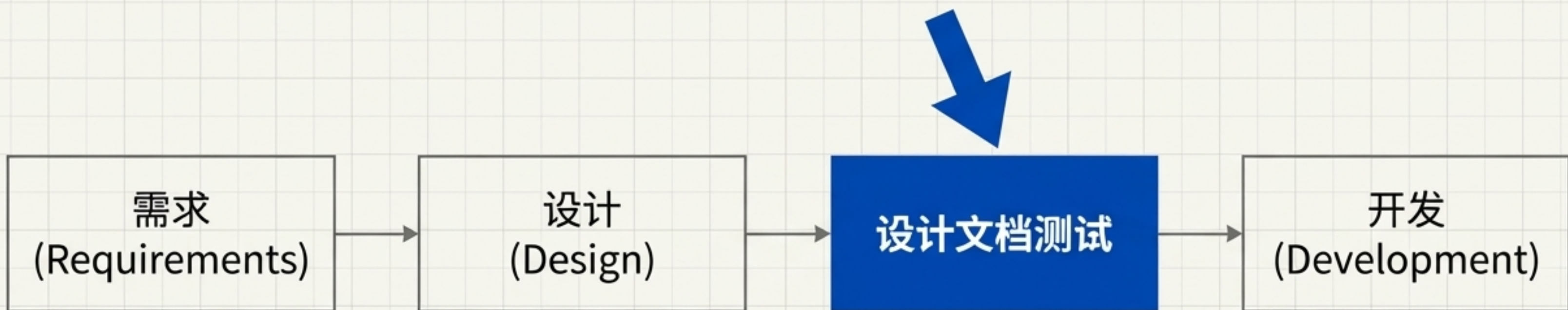


“对大模型来说，文档跟程序没有区别。”

人类工程师看到的是文字和逻辑图，大模型看到的是系统、状态和用户路径的抽象逻辑。

这种独特的理解能力，让一件过去不可能的事情成为现实：在设计阶段，像执行代码一样“执行”设计文档。

引入一个全新环节：设计文档测试



- 在设计文档完成之后、交付开发之前，引入的一个全新的、AI原生的测试环节。
- 它将最关键的验收工作，从开发周期的末端，前移到了起点。

这是真正的“测试”，而非“审核”

我们必须明确区分这两个概念，它们的性质有本质区别。



审核 (Review/Audit)

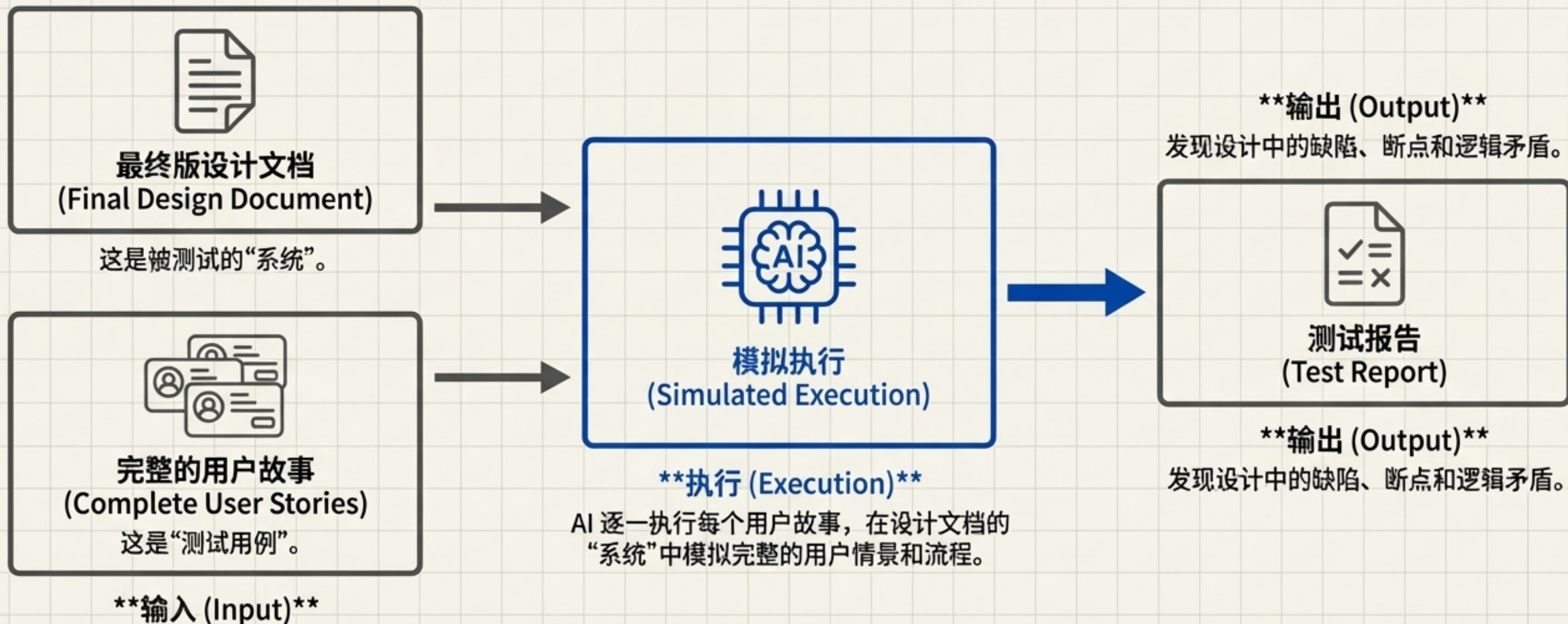
- **本质:** 主观检查 (Subjective Check)
- **方式:** 依赖人类经验、核对清单
- **目标:** 确保完整性、一致性
- **结果:** 意见、建议



设计文档测试 (Design Document Testing)

- **本质:** 客观执行 (Objective Execution)
- **方式:** 模拟用户场景、执行用户故事
- **目标:** 发现流程缺陷、逻辑错误
- **结果:** 通过/失败的测试报告

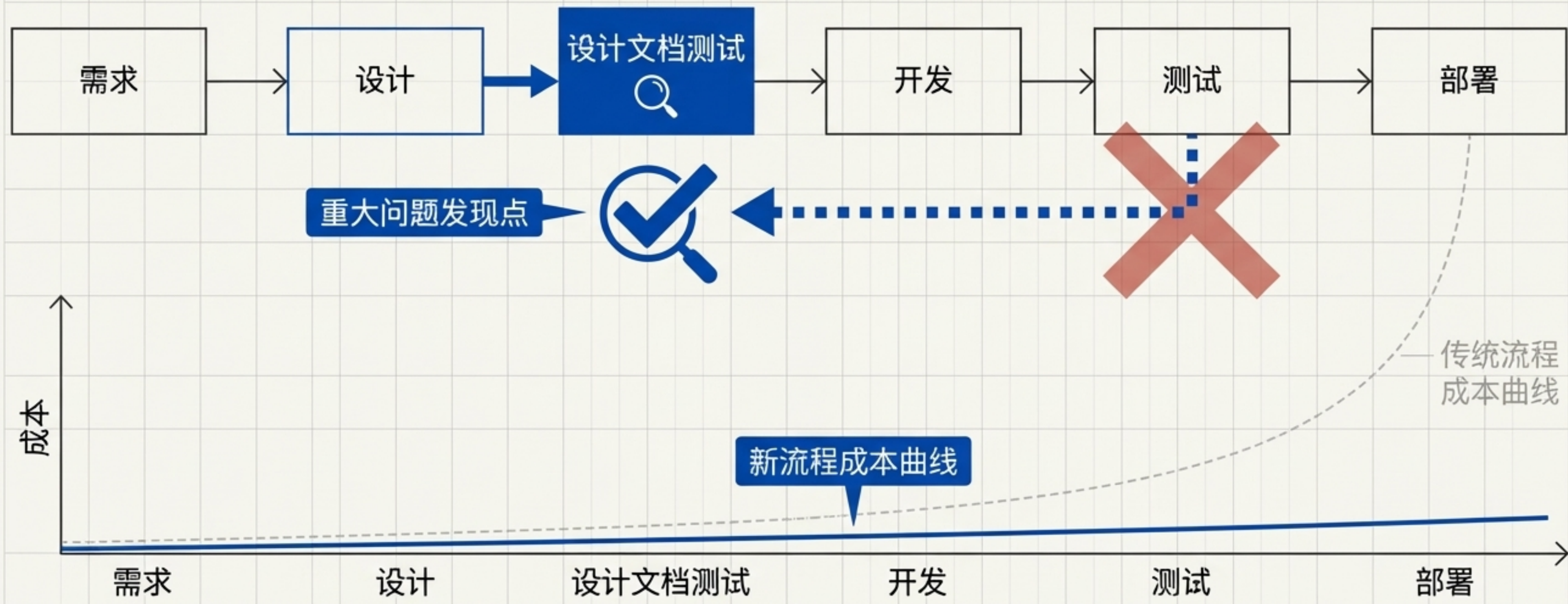
工作原理：将用户故事作为测试用例



这无限接近于用 UAT 的方式，对文档做最终产品级别的验收测试。

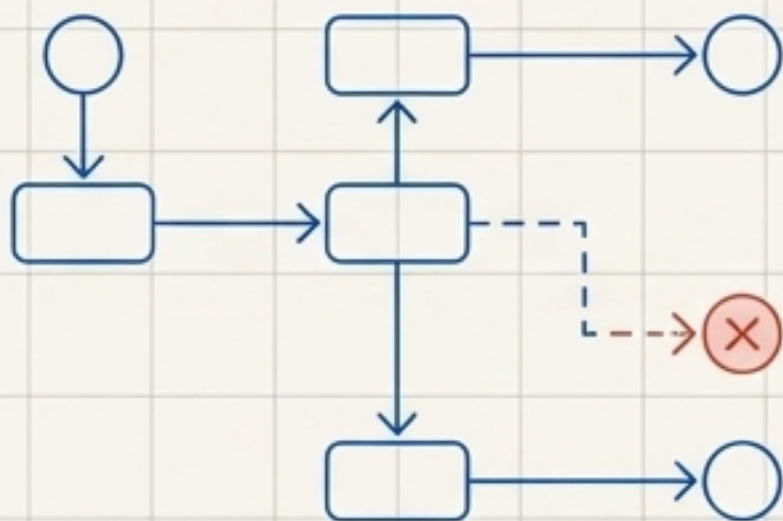
新的开发流程：在起点终结问题

通过设计文档测试，我们将重大问题的发现点从周期的末端彻底移到了起点。
这不仅仅是优化，这是对传统风险模型的颠覆。



在设计阶段，我们能发现什么？

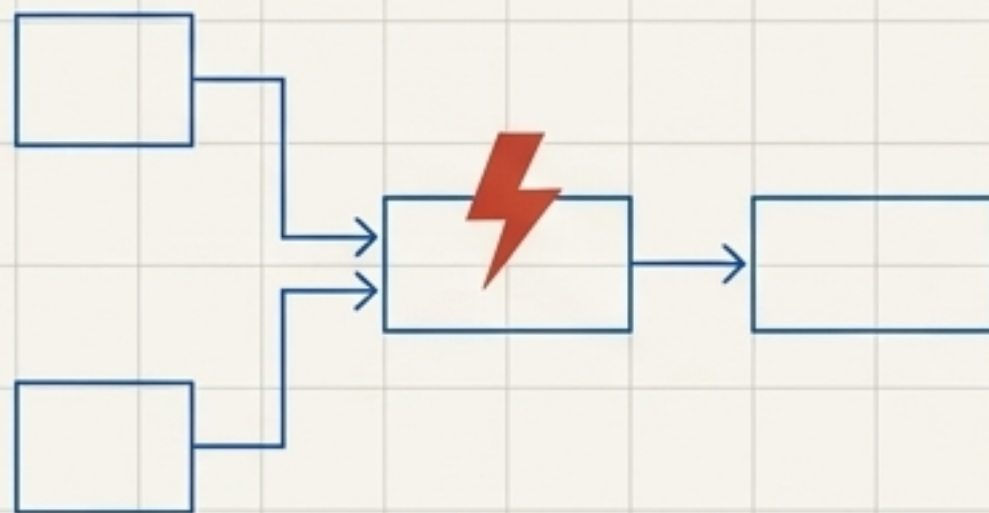
1. 用户体验 (UX) 缺陷 (UX Flaws)



所有与界面和流程相关的用户体验问题。

- 例如：不完整用户旅程、逻辑断点、交互死胡同、信息缺失等。

2. 架构与逻辑错误 (Architectural & Logical Errors)



程序架构中可能存在的运行时错误。

- 例如：状态管理冲突、数据流矛盾、权限逻辑漏洞等。

最终影响：让写代码成为一次性的简单流程

当所有UX、流程和核心逻辑问题在设计阶段被彻底解决后，编码阶段的目标就变得极其纯粹。

开发不再是“边写边改、边测边调”的探索过程。

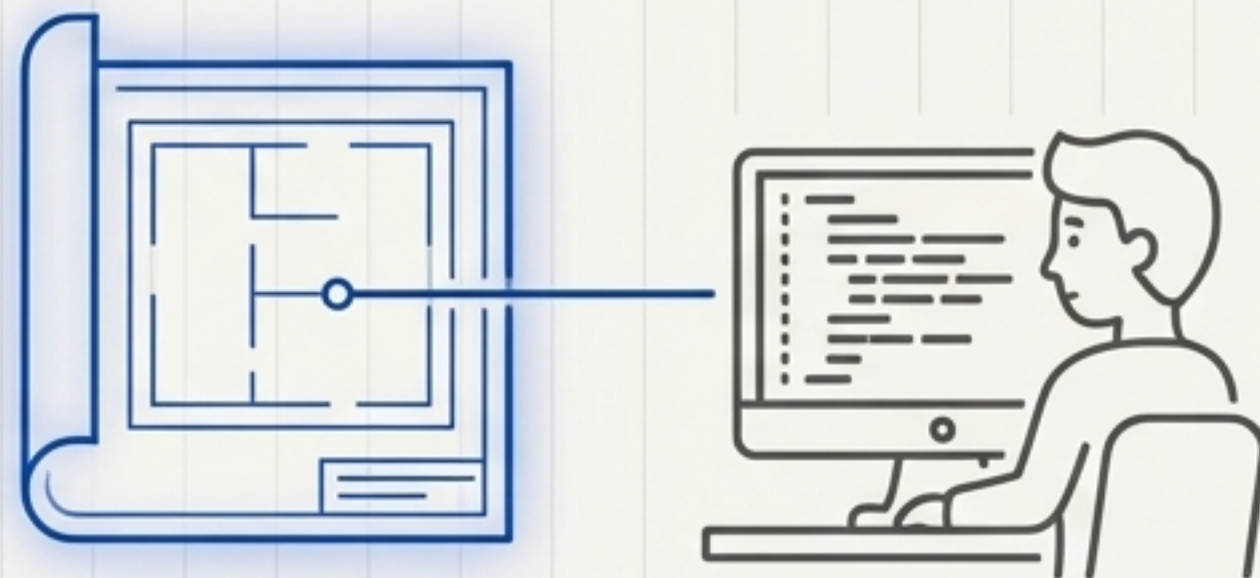
它回归了工程的本质：将一个已经过严密测试的、高质量的蓝图，精确地实现出来。

Before



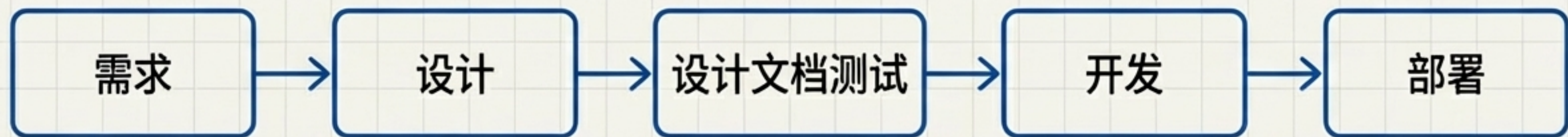
← 编码 = 探索 + 实现 →

After



编码 = 精确实现

欢迎来到 AI 原生开发的新时代



设计文档测试不是传统流程的改良，它是只有在 AI 原生时代才可能诞生的全新物种。
它从根本上改变了我们验证创意、管理风险和构建高质量软件的方式。

“这些都是AI原生的能力，传统开发流程不可能做得到，所以不存在。”